

159.102 Notes

Section 2 : Computer Memory and Number Systems

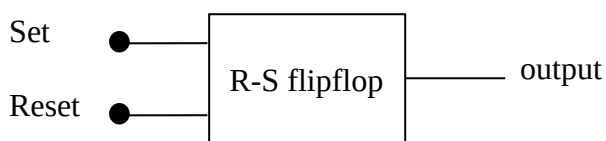
This section looks at how computer memory is constructed from circuits and how computers store integer values in memory. People usually work in the decimal number system but computers work in binary. Hexadecimal is used as a shorthand for writing binary.

The flipflop

The flipflop is a circuit with 2 inputs and 1 output. It is an **independent** circuit (it does not require continuous input) and it is a **bistable** circuit (it has 2 states). We could study the circuit required to construct a flipflop but we will not do that. If you are interested, you could look at:

[https://en.wikipedia.org/wiki/Flip-flop_\(electronics\)](https://en.wikipedia.org/wiki/Flip-flop_(electronics))

There are different types of flipflop – we look at the R-S flipflop which we represent as a black box:



The R-S flipflop works like this:

When Set is switched ON – the output is ON.

When Set is switched OFF – the output remains ON.

When Reset is switched ON – the output is OFF.

When Reset is switched OFF – the output remains OFF.

In other words, we use Set to make the output come ON and we use Reset to make the output go OFF.

We then use a code (ON = 1 and OFF = 0). This means that a flipflop is a “box” (or circuit) that can store a number, except that the number can only be zero or one. A computer that can only store the numbers 0 and 1 is rather limited, so we group flipflops together to form a **memory location**.

flipflop	flipflop	flipflop	flipflop	flipflop	flipflop	flipflop	flipflop
ON	OFF	OFF	ON	OFF	ON	ON	OFF
1	0	0	1	0	1	1	0

The numbers 0 and 1 are the **digits** used by the binary number system. A flipflop can store one binary digit (0 or 1) and so a flipflop is renamed a **bit** (short for **binary digit**). A group of 8 flipflops together is called a **byte**. Important – **there are 8 bits in one byte**.

Number Systems

Any number system has a **base** and a **set of digits** to represent the number. The digits are arranged in columns where each column represents a different **power** of the base. Number systems have been used for thousands of years – they have nothing to do with computers.

The Decimal number system

Let us start off by studying the decimal number system. For thousands of years, humans have used the decimal number system for counting. It is convenient to use the decimal number system because we have ten fingers. This is why the English word 'digit' means a number symbol and also means a finger (or a toe). However, not all societies have used the decimal number system. For instance, in about 3000BC, the Babylonians used the number system with base 60. And that is why we have 60 seconds in a minute and 60 minutes in an hour.

Important definitions:

digit – the symbols required to create a number in a particular number system. For example, the digits in the decimal number system are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. With these digits you can create any number that you need. **In the decimal number system there are ten digits** (decimal means ten).

number – a group of digits in a particular number system that represent a number. For example, the number 50 in the decimal number system has 2 digits (a 5 and a 0). It represents the number that comes after 49 and before 51.

The decimal number system collects numbers into groups of 10. The **base** of the decimal number system is 10. The **digits** of the decimal number system are 0 to 9. To represent any number we write down how many groups we have of each **power of 10**.

The powers of 10 are:

$1 (= 10^0)$ $10 (= 10^1)$ $100 (= 10^2)$ $1000 (= 10^3)$ and so on...

Important: we group the powers of 10 the other way around, i.e. 1 is always at the right of the number.

When we write a number we decide how many we need of each power of 10.

For example, the number 472 means that we have:

4 groups of 10^2 and 7 groups of 10^1 and 2 groups of 10^0

So when we write 472 we actually mean the result of the calculation: $4*100 + 7*10 + 2*1$

We should always write the base of the number system next to a number: 472_{10} (but we don't bother)

Adding decimal numbers

The groups of powers of 10 are very important when we add decimal numbers. We look at each power of 10 and add the digits in that group. If the result is ten (the base) or more, we carry one group of ten to the next power of 10.

For example: $28 + 36$

To add these numbers, we start with the digits in the group 10^0

So we have $8 + 6 = 14$ and 14 is greater than (or equal to) 10 so we break the 14 into one group of ten plus a 4. We leave the 4 in the 10^0 group and **carry the one** (group of ten) into the next power of 10.

Now we look at the digits in the group 10^1 and we include the 1 that we have just carried in.

So we have $2 + 3 + 1 = 6$.

The final result is: $6*10^1 + 4*10^0 = 6*10 + 4*1 = 64$ (Quick check – is 64 the result of $28 + 36$?)

We usually write an addition like this:

$$\begin{array}{r} 28 \\ + 36 \\ \hline 64 \end{array}$$

It is very important to remember that we **carry one** from the 10^0 column to the 10^1 column.

Here is another addition. Where do we have to **carry one** in this example?

$$\begin{array}{r} 99 \\ + 1 \\ \hline 100 \end{array}$$

The Binary number system

The binary number system collects numbers into groups of 2. The **base** of the binary number system is 2. The **digits** of the binary number system are 0 to 1. To represent any number we write down how many groups we have of each **power of 2**.

The powers of 2 are:

$1 (= 2^0)$ $2 (= 2^1)$ $4 (= 2^2)$ $8 (= 2^3)$ $16 (= 2^4)$ and so on...

Important: we group the powers of 2 the other way around, i.e. 1 is always at the right of the number.

When we write a number we decide how many we need of each power of 2.

For example, the number 101 means that we have:

1 group of 2^2 and 0 groups of 2^1 and 1 group of 2^0

So when we write 101 we actually mean the result of the calculation: $1*4 + 0*2 + 1*1 = 5$

We should always write the base of the number system next to a number: 101_2

Thus the number 101_2 represents the same value as the number 5_{10}

Adding binary numbers

The groups of powers of 2 are very important when we add binary numbers. We look at each power of 2 and add the digits in that group. If the result is two or more, we carry one group of 2 to the next power of 2.

For example: $101 + 100$

To add these numbers, we start with the digits in the group 2^0

So we have $1 + 0 = 1$ which is less than 2 so the result for 2^0 is 1

Then we have $0 + 0 = 0$ which is less than 2 so the result for 2^1 is 0

Then we have $1 + 1 = 2$ which is greater than or equal to 2 so we leave 0 in the group and **carry the one** (group of two) into the next power of 2. Thus the result for 2^2 is 0

Finally we have nothing (in the group 2^3) plus 1 (carried over) so the result for 2^3 is 1

Final result is: $1*2^3 + 0*2^2 + 0*2^1 + 1*2^0 = 8 + 0 + 0 + 1 = 9$

We usually write an addition like this:

$$\begin{array}{r} 101 \\ + 100 \\ \hline 1001 \\ \hline \end{array}$$

It is very important to remember that we **carry one** from the 2^2 column to the 2^3 column.

Here is another addition. Where do we have to **carry one** in this example?

$$\begin{array}{r} 111 \\ + 11 \\ \hline 1010 \\ \hline \end{array}$$

Carry 1 in binary

It is useful to know all the situations that can arise when adding binary digits:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ carry } 1$$

$$1 + 1 + 1 = 1 \text{ carry } 1$$

Bits and Bytes

Computers store numbers using the binary number system. A flipflop represents a binary digit and can be zero (OFF) or one (ON). In order to store larger binary numbers we use a collection of flipflops so that each flipflop represents a group in the binary number system.

For example, in order to store the number 9 in a computer we do the following:

Step 1: convert the number 9 from decimal to binary and we get the binary number 1001

Step 2: we need four flipflops because the number has four binary digits

Step 3: SET (or RESET) the four flipflops as follows: ON OFF OFF ON

Actually, the standard group of flipflops is 8 (8 bits in a byte) so we would use 8 flipflops set as follows: OFF OFF OFF OFF ON OFF OFF ON (or, in binary, 00001001)

An interesting thought

An interesting exercise is to calculate what is the maximum value that can be stored in a specified number of bits. Easy to do, just assume every flipflop is ON (i.e. assume every bit is 1).

E.g. maximum value you can store in 4 bits is: $1111 = 15$ (Fun fact: 4 bits is called a nibble).

As computers increased in power, they were able to include thousands of bytes in the memory. 2^{10} (1024) bytes is called a **kilobyte** (written as KB). It is sometimes assumed that a kilobyte is 1000 bytes if precision is not required. A **megabyte** is 1,000,000 bytes, etc.

Converting binary to decimal

Calculate the decimal value of a binary number by multiplying by the powers of 2. For example:

$$\begin{aligned}1101_2 &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\&= 8 + 4 + 0 + 1 \\&= 13_{10}\end{aligned}$$

Converting decimal to binary

Keep dividing by 2 until the number gets to zero. Keep track of the remainders. Then write the remainders down backwards. For example:

2	13
2	6 rem 1
2	3 rem 0
2	1 rem 1
	0 rem 1

Now write the remainders down backwards = 1101_2

Another way of writing the same conversion could be:

$$13 / 2 = 6 \text{ [rem 1]} \quad 6 / 2 = 3 \text{ [rem 0]} \quad 3 / 2 = 1 \text{ [rem 1]} \quad 1 / 2 = 0 \text{ [rem 1]}$$

Now write the remainders down backwards = 1101_2

The Hexadecimal number system

The hexadecimal number system collects numbers into groups of 16. The **base** of the hexadecimal number system is 16. To represent any number we write down how many groups we have of each **power of 16**. The **digits** of the hexadecimal number system are 0 to 9 and A, B, C, D, E, F. These can also be written as a, b, c, d, e, f. A represents the value 10, B = 11, C = 12, D = 13, E = 14, F = 15. Many programmers call the hexadecimal number system **hex**.

The powers of 16 are:

$$1 (= 16^0) \quad 16 (= 16^1) \quad 256 (= 16^2) \quad 4096 (= 16^3) \quad \text{and so on...}$$

Important: we group the powers of 16 the other way around, i.e. 1 is always at the right of the number.

When we write a number we decide how many we need of each power of 16.

$$\text{Example: } 26_{16} = 2 \cdot 16^1 + 6 \cdot 16^0 = 32 + 6 = 38_{10}$$

$$\text{Example: } AB_{16} = A \cdot 16^1 + B \cdot 16^0 = 160 + 11 = 171_{10}$$

Adding hexadecimal numbers

The groups of powers of 16 are very important when we add hexadecimal numbers. We look at each power of 16 and add the digits in that group. If the result is sixteen or more, we carry one group of 16 to the next power of 16.

For example: $59_{16} + 68_{16}$

To add these numbers, we start with the digits in the group 16^0

So we have $9 + 8 = 17$ and 17 is greater (or equal to) 16 so we break the 17 into one group of sixteen plus a 1. We leave the 1 in the 16^0 group and **carry the one** (group of sixteen) into the 16^1 column.

Now we look at the digits in the group 16^1 and we include the 1 that we have just carried in.

So we have $5 + 6 + 1 = 12$. We write 12 as C in hexadecimal.

The final result is: $12 \cdot 16^1 + 1 \cdot 16^0 = C1_{16}$

We usually write an addition like this:

$$\begin{array}{r} 59_{16} \\ + 68_{16} \\ \hline C1_{16} \end{array}$$

Converting hexadecimal to decimal

Calculate the decimal value of a hexadecimal number by multiplying by the powers of 16.

For example:

$$\begin{aligned} 2BF_{16} &= 2 \cdot 16^2 + B \cdot 16^1 + F \cdot 16^0 \\ &= 2 \cdot 256 + 11 \cdot 16 + 15 \cdot 1 \\ &= 703_{10} \end{aligned}$$

Converting decimal to hexadecimal

Keep dividing by 16 until the number gets to zero. Keep track of the remainders.

Then write the remainders down backwards. For example, convert 38_{10} to hexadecimal.

$$\begin{array}{r|l} 16 & 38 \\ 16 & 2 \text{ rem } 6 \\ & 0 \text{ rem } 2 \end{array}$$

Now write the remainders down backwards = 26_{16}

Converting binary to hexadecimal (IMPORTANT)

It is very easy to convert binary to hexadecimal by looking at groups of 4 binary digits:

For example, convert the binary number 100110_2 to hexadecimal.

Put the binary into groups of four (place extra zeros at the front if needed).

Thus 100110 becomes $0010\ 0110$

Now write the single corresponding hexadecimal digit for each group of four.

So 100110_2 becomes 26_{16}

We use hexadecimal as a shorthand for writing binary.

The Octal number system (this has NOTHING to do with our course)

Octal is the number system with base 8 and it uses the digits 0 to 7.

Example: $35_8 = 3 * 8^1 + 5 * 8^0 = 24 + 5 = 29_{10}$

Example: Add 42_8 and 27_8

$$\begin{array}{r} 42_8 \\ + 27_8 \\ \hline 71_8 \end{array}$$

Convert all three numbers to decimal and check the answer

We can use octal as a shorthand for binary by grouping binary digits in threes.

Example: Convert 29_{10} to binary:

2		29
2		14 rem 1
2		7 rem 0
2		3 rem 1
2		1 rem 1
2		0 rem 1

Now write the remainders down backwards = 11101_2

Group in threes and convert to octal = $011\ 101 = 35_8$

Octal was used for the same reason as hexadecimal, i.e. it is used as a shorthand to represent binary. Because a byte is 8 bits (i.e. 2 groups of 4 bits), hexadecimal is far more likely to be used than octal. However there are a few systems and organisations that still use octal. You do NOT have to know about the octal system.

How a computer represents negative integers

Computers have a **fixed word size**. This is the number of bits that are available to store one integer value. For example, in a computer with a word size of 8, there will be 8 bits used to store one binary number. So the number 5 (in such a computer) will be stored as: 0000 0101

In a computer with a word size of 16, the number 5 will be stored as: 0000 0000 0000 0101

We need to represent negative numbers and we use the fact that computers have a fixed word size.

Method 1: Sign and Magnitude

The **most significant bit** (at the left hand side of the number) becomes the **sign** bit.

The most significant bit = 0 for a positive number and 1 for a negative number.

For example: a computer with a word size of 6 will store 9 as 00 1001 and -9 as 10 1001

Problems with this method of representing negative numbers:

(a) we have to remember that the most significant bit is the sign – it is not part of the number.
This effectively reduces the number of available bits to store the number.

(b) $5 + (-5) = ?$

$$\begin{array}{rcl} 5 & = & 00\ 0101 \\ +\ (-5) & = & \underline{10\ 0101} \\ & & 10\ 1010 \end{array} \quad \text{This is } -10 \text{ but it should be zero.}$$

Let's move on to a different way of storing negative numbers that solves this problem...

Method 2: Ones Complement

Complement the number (i.e. switch zeros to ones and ones to zeros).

For example: a computer with a word size of 6 will store 5 as 00 0101 and -5 as 11 1010

Note that the **most significant bit** is 1 for -5 so we can still talk about a **sign bit**.

Example: calculate $4 + (-7)$ using a word size of 6 bits and ones complement arithmetic

$7 = 00\ 0111$ so -7 is the complement which is $11\ 1000$. **Note that the sign bit is a 1.**

$$\begin{array}{rcl} 4 & = & 00\ 0100 \\ +\ (-7) & = & \underline{11\ 1000} \\ & & 11\ 1100 \end{array} \quad \text{What is this number? – we know that it is negative.}$$

Complement this result to get 00 0011 which is the number 3.

Note that the complement of 3 is -3 and the complement of -3 is 3. Therefore the result was -3.

Now what happens if we calculate $5 + (-5)$ using a word size of 6 bits and ones complement?

$$\begin{array}{rcl} 5 & = & 00\ 0101 \\ +\ (-5) & = & \underline{11\ 1010} \\ & & 11\ 1111 \end{array} \quad \text{This number is } -0. \text{ What does that mean?}$$

Problem with this method of representing negative numbers:

We have two numbers: $0 = 00\ 0000$ and $-0 = 11\ 1111$

Is the following true or false: `if (0 == -0) {`

In a computer using ones complement, this test would be false and this could cause problems.

Method 3: Twos Complement

Complement the number **and add 1**.

We do not attempt to reverse this process. To go from positive to negative we complement and add 1. And to go from negative to positive we also complement and add 1. Note that every negative number has a leading bit of 1 so we can still talk about **the sign bit**.

Example: assume a word size of 6 bits:

$$\begin{array}{rcl} 5 & = & 00\ 0101 \text{ and then } -5 = 11\ 1010 \quad (\text{complement}) \\ & & + \quad \underline{\quad 1 \quad} \quad (\text{add 1}) \\ & & 11\ 1011 \quad (\text{this is } -5) \end{array}$$

Example: let us calculate $6 + (-4)$ using a word size of 6 bits and twos complement arithmetic:

Note: $4 = 00\ 0100$ and $-4 = 11\ 1011 + 1 = 11\ 1100$

$$\begin{array}{rcl} 6 & = & 00\ 0110 \\ + (-4) & = & \underline{11\ 1100} \\ 1 & 00\ 0010 & \text{The answer is } 2 = 00\ 0010. \end{array}$$

Note that there was a **carry of 1** but this cannot be stored because the word size is 6 bits. Therefore the extra 1 is simply ignored and is not part of the number.

Now what happens if we add $5 + (-5)$?

$$\begin{array}{rcl} 5 & = & 00\ 0101 \\ + (-5) & = & \underline{11\ 1011} \\ 1 & 00\ 0000 & \text{Now we get the correct answer of zero.} \end{array}$$

Note that 1 is carried out and cannot be stored because the word size is 6 bits. Therefore the extra 1 is simply ignored and is not part of the number.

Thus $5 + (-5) = 0$ which is the desired result.

All computers now use twos complement to represent negative integer values.

Definitions – you need to know these

fixed word size – the number of bits that are available to store an integer value in a particular computer. For example, in a computer with a word size of 8, there will be 8 bits used to store one binary number. So the number 7 (in such a computer) will be stored as: 0000 0111

most significant bit – the bit at the left-hand-side of a binary number in a computer. Thus for the number 01111 (assuming a word size of 5), the most significant bit is zero. Also called the **sign bit**.

complement – switch all the ones to zeros and all the zeros to ones in a binary number.

carry – if two bits are added producing a result greater than or equal to 2, then a 1 is carried to the next bit on the left. No carry is possible when adding the most significant bits of two numbers.

twos complement – a system that represents the negative of a number as “complement and add 1”. Used by virtually every computer in the world to represent negative integer values.

Largest integer in a computer

When we declare variables in a C++ program, the computer sets aside a specified amount of storage in memory. This storage space is a fixed number of bits, depending on the data type of the variable.

Example using char:

```
char value;
```

This sets aside 8 bits (1 byte) in the memory. Note that we may think of char as a character but it is actually a number. We can then use the variable in the program, for example:

```
value = 65;
```

This loads 65 into the variable. Because this variable is a char (8 bits) the computer loads the binary number representing 65 into the 8 bits, like this:

```
0100 0001
```

Question: what is the largest integer value that can be stored in a char variable (8 bits)?

Answer: $0111\ 1111_2 = 127_{10}$

Note that the first bit is a zero because it is not part of the number. It is the sign bit.

General rule: if you have n bits then the largest integer is $2^{n-1} - 1$

Example using short:

```
short num;
```

This sets aside 16 bits (2 bytes) in the memory. We can then use the variable in the program, for example:

```
num = 13;
```

This loads 13 into the variable. Because this variable is a short (16 bits) the computer loads the binary number representing 13 into the 16 bits, like this:

```
0000 0000 0000 1101
```

Question: what is the largest integer value that can be stored in a short variable (16 bits)?

Answer: $0111\ 1111\ 1111\ 1111_2 = 32,767_{10}$

Note that the first bit is a zero because it is not part of the number. It is the sign bit.

Example using int:

```
int count;
```

This sets aside 32 bits (4 bytes) in the memory. We can then use the variable in the program, for example:

```
count = 26;
```

This loads 26 into the variable. Because this variable is an int (32 bits) the computer loads the binary number representing 26 into the 32 bits, like this:

```
0000 0000 0000 0000 0000 0000 0001 1010
```

Question: what is the largest integer value that can be stored in an int variable (32 bits)?

Answer: $0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = 2,147,483,647_{10}$

Note that the first bit is a zero because it is not part of the number. It is the sign bit.

Note: there are other variable types:

We have looked at char, short and int. There are other data types for integer values such as long. Each variable type has a number of bits associated with it. The number of bits can change over time. For example, during the 1990's an int usually had 16 bits allocated to it.

The number of bits used to store an int is called the word size. During the 1990's most computers had a word size of 16 bits. Now most computers have a word size of 32 bits or 64 bits. This comes up when you want to download some applications. It may say "do you want to download the 32 bit version or the 64 bit version?" You have to know the word size of your computer (and your system) in order to make the best selection.

Program to check the size of a variable

Here is a useful little program to check the size of a variable and to test the maximum value that can be stored:

```
#include <iostream>
using namespace std;

short value;      // try this with short or char or int
int numbits;

int main() {
    cout << "Enter a value ";
    cin >> value;
    cout << "The value is " << value;
    numbits = sizeof(value) * 8;
    cout << "\nThe size of the variable is " << numbits << endl;
    value = value + 1;
    cout << "\nPlus 1 = " << value << endl;
}
```

sizeof is a function that returns the number of bytes (multiply by 8 to get bits) in a variable.

The last two lines of the program are to see what happens when you add 1 to the largest number that can be stored. For example, define the variable as short, then enter 32767 and see what is displayed.

maxint (IMPORTANT for all programmers)

maxint is the largest integer value that can be stored in a computer. The size of maxint will differ, depending on the variable used (e.g. long) and the make of computer BUT there is always a maxint. It is very important for programmers to know about maxint. In other words, you must know that there is a particular integer value that if you add 1 (or greater) to it, you will get a meaningless answer. Many programmers create programs without realising that they may set up a series of calculations that could produce a meaningless result (if the final result is greater than maxint).